DAY 2 · MODULES 3 & 4 · PRE-READ

Day 2 · Modules 3 & 4

Module 3, Verify Your Product Metrics (Rewind), and Module 4, Inspect Codebases Without Coding (X-Ray Vision).

Claude Code for PMs · **Pre-Read** · **Modules 3 & 4**

SESSION 1

Module 3 · Rewind

Verify Your Product Metrics · superpower: “Rewind”

Ask the data directly, then check whether the answer is credible

15 min. Skim before the live session.

You do not write or read any code in this course. You work inside your own repo, created from the course template in Module 1.

Where we are

You're the new PM at Rook Industries, which builds software for actual superheroes. Your product is Rook Dispatch, which gets a superhero to where they're needed when there's an emergency. It works out who is close enough and free enough to help, and gets hold of them. Release 4.2 shipped on August 12, shortly before you arrived, and it has problems.

Where things stand. You joined Rook two weeks ago as PM on Dispatch. Release 4.2 shipped on 12 August, just before you arrived, and landed badly. Last session you read four interviews and every support ticket since, and found the two piles didn't agree. Today a third source lands on your desk.

The technique this module turns on

Ask for the number. Then ask to see the rows.

1. **Don't just ask what's the number** That gets you a single figure with nothing behind it. You can't tell if it's right, and you can't check it against anything.
2. **Ask for the rows behind it** Now the number traces back to something real: the actual weeks, the actual responder, the actual pings.
3. **Then work it out a second way** If the two answers agree, you can trust it. If they don't, you just found something worth a meeting, in ten seconds instead of during one.

Why this matters for Claude Code. This isn't just a better question, but something only Claude Code can actually deliver. It doesn't answer from a chat's memory of a number someone told you. It reads the real rows in the pings table through rook-database every time, so a figure traces straight back to rows you can reopen. And you won't be doing any of the maths yourself: Claude does that. Your job is the harder one, asking a question worth answering, and noticing when the answer looks wrong.

What you'll do in the session

3 hands-on labs, 68 minutes of lab time in total.

1. **Lab A: What Happened After Release 4.2** (28 min), Three steps: ask the pings table, push past the first answer, then post the number you'd actually defend.
2. **Lab B: Does the Story Hold Up?** (20 min), Two steps: put the two sources side by side, then check the most confident thing anyone's told you.
3. **Lab C: One Person's Story** (20 min), Three steps: pick someone, follow their month, then say it in plain English.

What lands in your repo

`03-rewind/prompts.md`, the prompts you wrote yourself. Claude Code writes the file at the end of the session; you never open it by hand. Commit it before you close.

SESSION 2

Module 4 · X-Ray Vision

Inspect Codebases Without Coding · superpower: “X-Ray Vision”

Debug the code without coding

15 min. Skim before the live session.

You do not write or read any code in this course. You work inside your own repo, created from the course template in Module 1.

Where we are

You're the new PM at Rook Industries, which builds software for actual superheroes. Your product is Rook Dispatch, which gets a superhero to where they're needed when there's an emergency. It works out who is close enough and free enough to help, and

gets hold of them. Release 4.2 shipped on August 12, shortly before you arrived, and it has problems.

Where things stand. A few weeks in now, and Rook feels less like a black box. Last session confirmed the timing changed, ninety seconds down to sixty, and it landed on real people. What you don't have yet is why that mattered enough to make somebody stop hearing from us completely.

The technique this module turns on

How to Read Any Codebase

Ask about the code, not how to read it.

```
import logging


class NotificationDispatcher:
    def __init__(self, retry_limit=3):
        self.retry_limit = retry_limit
        self.logger = logging.getLogger(__name__)

    def send(self, user, message):
        for attempt in range(self.retry_limit):
            if self._deliver(user, message):
                return True
            self.logger.warning(f"Retry {attempt+1} for {user.id}")
        return False
```

1. **Ask what it does, start to finish** Describe the whole thing in plain English, the way you'd explain your own product to somebody outside the room.
2. **Ask it to translate anything unfamiliar** Any term that trips you up is a question, not a wall. Keep asking until the answer makes sense to a PM.

3. **Then ask what it doesn't do** Code only shows you what somebody built. Ask what's missing and you'll find the gap an engineer would only ever notice by accident.

Why this matters for Claude Code. This is the same instinct as running a good interview or mapping a user journey: knowing what to ask, and noticing when the answer is quietly incomplete.

What you'll do in the session

3 hands-on labs, 68 minutes of lab time in total.

1. **Lab A: A First Look at 4.2's Code** (24 min), Three steps: point Claude at code you've never opened, push past the first explanation, then post a read you can defend twenty minutes in.
2. **Lab B: Check It Against What You Know** (20 min), Three steps: a question that's been sitting on the 4.2 release page for a week, a prompt you write yourself to test it, then the answer the engineering manager never got.
3. **Lab C: What the Code Doesn't Do** (24 min), Three steps: find what takes points off, push it further, then say it plainly.

What lands in your repo

`04-x-ray-vision/prompts.md`, the prompts you wrote yourself. Claude Code writes the file at the end of the session; you never open it by hand. Commit it before you close.

DAY 2 · MODULES 3 & 4 · PRE-READ · CLAUDE CODE FOR PMS

Keep this open while you build.

Claude Code for PMs · Reference companion